Doctoral Symposium: Large-Scale Stream Graph Processing

Domagoj Margan
Imperial College London
180 Queen's Gate
London, United Kingdom SW7 2AZ
d.margan15@imperial.ac.uk

Peter Pietzuch
Imperial College London
180 Queen's Gate
London, United Kingdom SW7 2AZ
prp@doc.ic.ac.uk

ABSTRACT

Dynamically changing graphs are a powerful abstraction used to represent temporal relationships and connections occurring between data entities in various real-world organizations, such as social and telecommunication networks. The increasing volume, variety and velocity of graph-structured data in many application domains have led to a development of large-scale graph processing systems. However, current state-of-the-art graph processing systems do not provide efficient support for streaming graph scenarios. In this report, we describe and discuss *stream graph processing*, which narrows the problem of traditional graph processing by focusing on near real-time analysis of dynamic graph data constructed and maintained from stream sources, as opposed to processing of historical graph datasets loaded from a disk storage. We provide an outline of challenges in stream graph processing and present our preliminary approach to designing a stream graph processing system done as a part of early PhD work.

CCS CONCEPTS

• **Information systems → Data streaming**; **Data stream mining**;

KEYWORDS

graph processing, graph mining, dynamic graph, temporal graph, stream graph

1 INTRODUCTION

The usage of graph-structured data is increasing in all areas of data processing. Graphs are commonly adapted to represent connections and relationships existing in real-world organizations, such as social networks and telecommunication networks. Vertices represent data entities, such as users, telecommunication devices, or websites, while edges represent connections and interactions occurring between them.

In addition to increasing size and complexity, information flow in real-world networks is highly dynamic - data is continuously generated as networks are evolving. Major social networks are constantly experiencing massive growths in their user base, as well as in a number of daily active users and activities. For example, Facebook reports [12] 1.15 billion of daily active users and 1.8 billion of monthly active users with a yearly increase of 17 percent. That leads to an average of 8500 new comments and 4800 statuses being posted on Facebook every second. Similarly, an average of 313 million monthly active Twitter users is creating 500 million tweets per day or 6000 tweets per second [13].

Such time-evolving real-world networks can be modeled as *dynamic graphs* (also known as *temporal graphs*), where changes in graph data mirror the dynamics of organizations they represent, most commonly in a form of addition, removal, or modification of vertices and edges.

Extracting information and knowledge in near real-time from streaming graph-structured data can be beneficial for industry and businesses, as well as for gaining scientific and social insights. In order to achieve such goals, we must design and develop an efficient large-scale stream processing system optimized for dynamic graph data. That is the main aim of the PhD work presented in this report.

We first discuss general workflow and workload characteristics of stream graph processing, as well as the design and implementation challenges a stream graph processing system must address. We furthermore present our preliminary approach to designing a stream graph processing system done as a part of early PhD work. Lastly, we discuss some possible future directions and conclude this report.

2 STREAM GRAPH PROCESSING

Formally, a *dynamic* (or *temporal*) *graph* T can be defined as a set of graphs $T = \{G_1, G_2, G_3, ..., G_t\}$ where each $G_i \in T$ is a static snapshot of T at a time i. Each $G_i = (V_i, E_i)$ where V_i represents the set of vertices and E_i represents the set of edges between any $u, v \in V_i$ at a time i. A dynamic graph can be *vertex-dynamic* if the vertices are added, modified or removed from the graph during time, and *edge-dynamic* if the edges are added, modified or deleted from the graph. A graph can be both vertex- and edge-dynamic [5].

Dynamic graph processing is a new and exciting systems research topic. Although the idea and application of dynamically changing graphs in computer science are at least several decades old [5], first systems specifically oriented at dynamic graph processing emerged recently driven by the rise of big data and social networks [1, 4]. Generally speaking, such systems aim to provide efficient computational frameworks and mechanisms for management and analysis of large-scale dynamic graph data.

Stream graph processing described in this work narrows the problem of dynamic graph processing by focusing on near real-time analysis of streaming graph data, as opposed to processing historical dynamic graph datasets from disk storage. Current state-of-the-art dynamic graph processing systems [4, 9] assume pre-availability of historical graph datasets stored on disks as they require expensive

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).
DEBS '17, June 19-23, 2017, Barcelona, Spain
© 2017 Copyright held by the owner/author(s).
ACM ISBN 978-1-4503-5065-5/17/06…$15.00
https://doi.org/http://dx.doi.org/10.1145/3093742.3093907

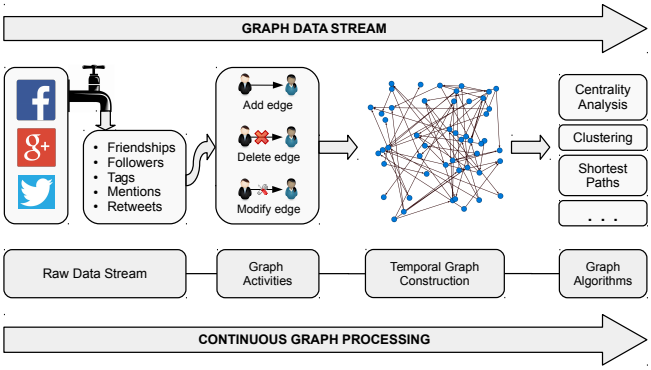

Figure 1: An illustration of a stream graph processing workflow.

data preprocessing, thus they are not suitable for near real-time stream processing.

In this section, we provide an overview of a stream graph processing workflow as well as a list of design and implementation challenges.

2.1 Workflow

In contrast to the traditional graph processing model where a graph dataset is first stored and indexed and then subsequently processed by a given graph query, a stream graph processing system takes the inbound data while it is in flight, as it streams through the server.

A system captures raw stream data and parses it as *graph activities*. Graph activities are the most basic building blocks of an underlying dynamic graph: *a)* vertex additions, *b)* vertex deletions, *c)* vertex modifications, *d)* edge additions, *e)* edge deletions, and *f)* edge modifications. As such, graph activities mirror events happening in real-world networks, such as creation or deletion of a user profile on a social network, addition (following) or removal (unfollowing) a friend, and retweeting or mentioning users in tweets or posts.

According to [15], a dynamic graph can be constructed in *discrete* or *continuous* modeling frameworks. In a discrete graph framework, static graph snapshots are constructed periodically after a specified time period (e.g. every 10 minutes, every day, every week...). In other words, discrete framework captures a static state of a graph specifically at a defined time point. In that case, graph's historical changes are represented as a sequence of static sequential snapshots, where each snapshot represents dynamic graph's state at a single point in time. In contrast, the continuous graph modeling framework dynamically represents every activity occurring in a graph by mapping every individual change into a genuine graph state. Continuous graph modeling implies the preservation of the latest graph snapshot as well as the snapshots of interest while the graph's historical changes between any two consecutive graph snapshots of interest are written in a log file.

The constantly updating dynamic graph is periodically processed over windows of input data by specified queries or graph algorithms of interest, producing timely near real-time computation results. Figure 1 illustrates a described stream graph processing workflow using popular social networks as the source of streaming data.

2.2 Challenges

Maintaining near real-time processing times while managing a high rate of streaming updates in large-scale dynamic graphs is a demanding problem. The temporal dimension present in dynamic graphs introduces new challenges previously not applicable to traditional static graph processing systems. While the application and analysis of dynamic graphs could enable richer insights and continuous information mining in comparison to static graph datasets, the streaming model imposes several non-trivial design and implementation challenges.

In addition to well-known challenges recognized by state-of-the-art static graph processing systems [3, 6–8, 10, 11], a stream graph processing system needs to address further issues related to timeliness of computations, graph updating and maintenance, flexibility to data and workload changes, and ability to execute standard graph-mining algorithms.

Timely computations. A system must support scheduling of computations that span multiple snapshots in order to provide iterative execution of graph algorithms and queries in near real-time. High performance and high throughput computation must be assured in order to timely handle both large-scale and temporal aspects of streaming data. A system must support parallel computations on multicore or distributed environments.

In-memory dynamic graph maintenance. A system must provide an effective in-memory layout for storage and maintenance of dynamic graphs and corresponding snapshot series. A system must cope with a continuous large-volume stream of updates and efficiently manage an in-memory dynamic graph data structures. Due to its size and complexity, the storage and maintenance of real-world network data are expensive even if the date represents a single point in time. Effective maintenance of dynamic graphs requires flexible in-memory data layouts and optimized data structures.

Flexibility. A system must be capable of adapting to constant changes in the underlying graph topology, edge and vertex attributes, and type of graph processing query or load required by end users. More precisely, a system should cope with the following types of changes [14].

Topological changes: The vertices (e.g. social network users who were active during some defined time period) or edges (e.g. interaction between users on a social platform) that exist in the graph can change, modifying the graph topology and structure over time. Although the evolution of real-world networks is commonly steady paced [4], changes in organizations such as social networks can occasionally be fast and bursty [2]. Such changes in the graph topology and structure can create communication overheads and affect load balancing between machines or processing units.

Attribute changes: The information contained in a dynamic graph is constantly changing. The speed of the information dynamics depends on the underlying graph modeling and application. Changes

in information contained in a graph will cause the previously conducted computations to be obsolete, therefore systems need to update information on a near real-time basis.
Workload changes: The amount and type of queries on the dynamic graph may change according to the different seasonal culminations. As large graphs are commonly partitioned and distributed over several machines, changes in the workload can impact performance of the system by creating system hotspots.

Graph-mining algorithms. Most of the standard graph-mining algorithms assume a static graph as the input. A system should support standard graph-mining algorithms that extract insights from graph structure in combination with the temporal dimension of the underlying data.

The points in question described above should be addressed in both design and implementation of a system. In the following section, we present a step towards such a system done as a part of the early PhD work.

3 DESIGN AND IMPLEMENTATION

In this section, we first provide an outline of our stream graph processing system and a general description of its preliminary design. We then focus on some of the system's most important components and functions.

3.1 General overview

Our system consists out of three main system components: *ingest engine*, *activities-to-graph engine*, and *processing engine*. A design overview of our system is illustrated in the Figure 2.

Our system starts by connecting its *ingest engine* to a raw data stream, presumably sourced from an online social platform of interest. It connects to a raw data stream via specified API and continuously parses the events detected in the stream in order to capture dynamic graph activities. The process of parsing graph activities from raw data stream can vary based on a context of an underlying application.

Graph activities parsed and defined by the ingest engine are then continuously and individually sent to *in-memory activities-to-graph engine* which constructs continuous graph snapshots. A *graph snapshot* is a static image of a graph constructed from all graph activities which occurred in a data stream in a corresponding period of time. Thus, our system models a dynamic graph using the discrete modeling framework.

The system constructs multiple successive graph snapshots that represent graph activities which occurred in different consecutive periods of time in a data stream. Constructed graph snapshots are congregated in *snapshot batches* and are represented in a memory-efficient *time-locality layout* aware of temporal inter-relationships that exist between the snapshots. A snapshot batch can contain two or more individual graph snapshots - a size of the batch is flexible and determinable by memory limitations. Graph snapshots and graph batches are continuously updated while receiving new arriving graph updates from the ingest engine.

Once finalized, a dynamic graph captured in a snapshot batch of a given size is sent to the *processing engine*. Our system simultaneously keeps two snapshot batches in memory: one batch currently being constructed and updated, and one batch being processed by

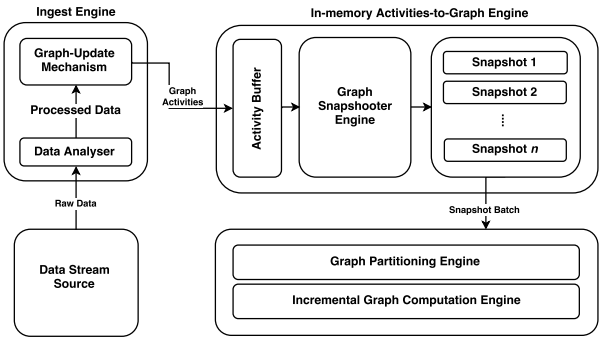

Figure 2: A simplified design overview of a stream graph processing system.

the processing engine. The main aim of the processing engine is to process a received snapshot batch by executing a computation of a given graph mining algorithm on each of the graph snapshots contained in a batch, incrementally and in parallel. Prior to the graph mining computations, a dynamic graph represented by a snapshot batch is partitioned so that the vertices from belonging snapshots are assigned to different CPU cores according to a given partitioning rule. Partitioning of graph snapshots enables efficient parallel computation. After the graph-mining computations are finished for the current snapshot batch, our system frees the memory by deleting needless batch data and enables the next snapshot batch to be processed.

Each of the system's individual components (ingest engine, activities-to-graph engine, processing engine) operates continuously and in parallel. The ingest engine constantly listens to a raw data stream and parses graph activities while the activities-to-graph engine is constructing a dynamic graph formed as a snapshot batch. Simultaneously and in parallel, the processing engine is executing a given graph mining algorithm on a previously constructed snapshot batch. The final output of our system is a continuous stream of results produced by a given graph mining algorithm.

3.2 Dynamic graph memory layout

A sequence of graph snapshots contained in a snapshot batch has two dimensions: a *graph-structure dimension* that represents interactions between neighboring vertices within a single snapshot and a *time dimension* that represents temporal evolution across all snapshots [9]. Our system models a graph batch data structure with a focus on a time dimension rather than on a graph-structure dimension when structuring a dynamic graph in memory.

The *time-locality* memory layout introduced by Han *et al.* [4] groups graph data in memory according to the time dimension in such a way that the states of the same vertex across multiple consecutive snapshots are laid out consecutively next to each other. As shown in [4], time-locality memory layout decreases the data access volumes and introduces potentially superior design choices for parallel graph computation in comparison to the layout based on the graph-structure dimension.

Our system's snapshot batch data structure utilizes time-locality memory layout and groups dynamic graph data by vertex, placing

close together different temporal versions of the same vertex across snapshots. Our snapshot batch data structure is formed out of two main sub-structures: *edge array* and *vertex data array*.

In the *edge array* edges are uniquely identified by the vertices they connect. All the edges are grouped by their source (or destination) vertices. Each entry in the edge array corresponds to a unique edge that can be present in one or more snapshots. Each edge in the edge array is joined with a matching snapshot bitmap which identifies all snapshots that contain that particular edge. Edges may have associated weight, and in that case, a snapshot bitmap would be formatted as a map of integer weights (instead of boolean values), where weight values would indicate the existence of an edge in particular snapshots.

Vertex data array is a structure that stores the data processing values for all vertices across all snapshots. It is utilized in vertex-oriented graph-mining procedures as a simple way to store and read results and values created during a computation.

3.3 Computation Model

Our system utilizes a modified version of the *vertex-oriented* computation scheme inspired by traditional graph processing systems [7, 8] in which graph-mining algorithms compute by processing across all vertices contained in a snapshot batch and by using vertex-associated data. Our system iteratively executes a given graph-mining algorithm, such as PageRank, over vertices of a dynamic graph where each of the vertices is associated with its own data (usually a numerical value) that is used in computation. The vertex-oriented graph-mining algorithm is designed from the perspective of a vertex, receiving as input the vertex's data as well as data from neighboring vertices. Such graph-mining algorithm may be computed across vertices of the graph in synchronous or asynchronous fashion, depending on the algorithm. Computation ends after either a specified number of iterations or after a certain convergence criterion has been met.

Our system's computation mechanism supports incremental processing of graph snapshots and snapshot batches. *Sequential incremental* computation presents a simple method of scheduling successive computations for each individual graph snapshot where computation results are updated based on recent changes in the dynamic graph. The basic idea of the sequential incremental computation is to fetch the results generated in a previous snapshot and reuse them as starting values for the currently computing snapshot. By reusing already computed vertex values from previous snapshots, sequential incremental processing avoids computing graph-mining algorithms from scratch at an increasing cost. In sequential incremental computation approach, our system supports parallel processing by executing computations for each vertex in a given snapshot simultaneously and in parallel.

4 CONCLUSION

In this Doctoral Symposium report, we described and discussed *stream graph processing* as well as the design and implementation challenges a stream graph processing system needs to address. We presented an overview and a preliminary design of a stream graph processing system done as a part of the early PhD work. As shown, our system is still in a prototyping stage and the presented design and implementation should be considered as a work in progress. Our next research steps include the further development of our system as well as the experimental validation and comparison with existing alternative solutions.

ACKNOWLEDGMENTS

The support of the EPSRC Centre for Doctoral Training in High Performance Embedded and Distributed Systems (HiPEDS, Grant Reference EP/L016796/1) is gratefully acknowledged.

REFERENCES

[1] Raymond Cheng, Enhong Chen, Ji Hong, Aapo Kyrola, Youshan Miao, Xuetian Weng, Ming Wu, Fan Yang, Lidong Zhou, and Feng Zhao. 2012. Kineograph: Taking the Pulse of a Fast-Changing and Connected World. In *Proceedings of the 7th ACM European Conference on Computer Systems - EuroSys '12*. Association for Computing Machinery, Bern, Switzerland, 85–98. https://doi.org/10.1145/2168836.2168846

[2] Sabrina Gaito, Matteo Zignani, Gian Paolo Rossi, Alessandra Sala, Xiaohan Zhao, Haitao Zheng, and Ben Y. Zhao. 2012. On the Bursty Evolution of Online Social Networks. In *Proceedings of the ACM HotSocial '12*. Beijing, China, 0–7.

[3] Joseph E. Gonzalez, Reynold S. Xin, Ankur Dave, Daniel Crankshaw, Michael J. Franklin, Joseph E. Gonzalez, Reynold S. Xin, Ankur Dave, Daniel Crankshaw, Michael J. Franklin, and Ion Stoica. 2014. GraphX : Graph Processing in a Distributed Dataflow Framework. In *Proceedings of the 11th USENIX Symposium on Operating Systems Design and Implementation - OSDI '14*. Broomfield, CO, USA, 599–613.

[4] Wentao Han, Youshan Miao, Kaiwei Li, Ming Wu, Fan Yang, Lidong Zhou, Vijayan Prabhakaran, Wenguang Chen, and Enhong Chen. 2014. Chronos: A Graph Engine for Temporal Graph Analysis. In *Proceedings of the 9th ACM European Conference on Computer Systems - EuroSys '14*. Association for Computing Machinery, Amsterdam, Netherlands. https://doi.org/10.1145/2592798.2592799

[5] F. Harary and G. Gupta. 1997. Dynamic Graph Models. *Mathematical and Computer Modelling* 25, 7 (1997), 79–87. https://doi.org/10.1016/S0895-7177(97)00050-2

[6] Aapo Kyrola, Guy Blelloch, and Carlos Guestrin. 2012. GraphChi: Large-Scale Graph Computation on Just a PC. In *Proceedings of the 10th USENIX Conference on Operating Systems Design and Implementation - OSDI '12*. Hollywood, CA, USA, 31–46.

[7] Yucheng Low, Joseph E. Gonzalez, Aapo Kyrola, Danny Bickson, Carlos Guestrin, and Joseph M. Hellerstein. 2012. Distributed GraphLab: A Framework for Machine Learning and Data Mining in the Cloud. *Proceedings of the VLDB Endowment* 5, 8 (2012), 716–727. https://doi.org/10.14778/2212351.2212354 arXiv:arXiv:1204.6078v1

[8] Grzegorz Malewicz, Matthew H. Austern, Aart J.C. Bik, James C. Dehnert, Ilan Horn, Naty Leiser, and Grzegorz Czajkowski. 2010. Pregel: A System for Large-Scale Graph Processing. In *Proceedings of the 2010 ACM SIGMOD International Conference on Management of Data - SIGMOD '10*. Association for Computing Machinery, Indianapolis, Indiana, USA, 135–146. https://doi.org/10.1145/1807167.1807184

[9] Youshan Miao, Wentao Han, Kaiwei Li, Ming Wu, Fan Yang, Lidong Zhou, Vijayan Prabhakaran, Enhong Chen, and Wenguang Chen. 2015. ImmortalGraph: A System for Storage and Analysis of Temporal Graphs. *ACM Transactions on Storage* 11, 3 (2015), 1–34. https://doi.org/10.1145/2700302 arXiv:1005.3014

[10] Amitabha Roy, Laurent Bindschaedler, Jasmina Malicevic, and Willy Zwaenepoel. 2015. Chaos: Scale-out Graph Processing from Secondary Storage. In *Proceedings of the 25th ACM Symposium on Operating Systems Principles - SOSP '15*. Monterey, CA, USA, 410–424. https://doi.org/10.1145/2815400.2815408

[11] Amitabha Roy, Ivo Mihailovic, and Willy Zwaenepoel. 2013. X-Stream: Edge-centricGraph Processing using Streaming Partitions. In *Proceedings of the 24th ACM Symposium on Operating Systems Principles - SOSP '13*. 472–488. https://doi.org/10.1145/2517349.2522740

[12] Facebook Statistics. Accessed: 2017-05-19. https://zephoria.com/top-15-valuable-facebook-statistics/. (Accessed: 2017-05-19).

[13] Twitter Statistics. Accessed: 2017-05-19. https://www.talkwalker.com/blog/40-amazing-twitter-stats-to-inspire-your-2017-social-strategy. (Accessed: 2017-05-19).

[14] Luis M. Vaquero, Felix Cuadrado, and Matei Ripeanu. 2014. *Systems for Near Real-Time Analysis of Large-Scale Dynamic Graphs*. Technical Report. HP Laboratories. arXiv:1410.1903

[15] Aya Zaki, Mahmoud Attia, and Doaa Hegazy. 2016. Comprehensive Survey on Dynamic Graph Models. *International Journal of Advanced Computer Science and Applications* 7, 2 (2016), 573–582.